Congressional Database Testing

Comprehensive Test Suite Documentation

Frontend & Backend Testing

DEVELOPERS:

Stefan Slacska

Dimitri Luck

Avery Robinson

UNIVERSITY *of* NORTH CAROLINA
PEMBROKE

Congressional Database Testing
Comprehensive Test Suite Documentation
Frontend & Backend Testing
1. Introduction
1.1 Project Overview
1.2 Scope of Testing
2. Test Case Design and Development
2.1 Testing Approach Overview
2.2 Test Environment Setup
3. Test Execution and Results
3.1 Testing Approach
3.2 Testing Evidence
3.3 Test Execution Log
3.4 Detailed Failure Analysis
3.5 Test Evidence (Logs)
3.6 Screenshot Evidence
4. Backend Testing
4.1 Testing Architecture Overview
4.2 Controller Layer Testing (18 tests, 90% coverage)
4.3 Service Layer Testing (58 tests, 79% coverage)
4.4 Repository Layer Testing (65 tests)
4.5 Entity Layer Testing (75 tests, 98% coverage)
4.6 DTO and Mapper Testing (119 tests)
4.6.1 DTO Layer (64 tests, 70% coverage - improved from 15%)
4.6.2 Mapper Layer (55 tests, 89% coverage)
4.7 Integration Testing (14 tests)
4.8 Code Coverage Analysis
4.9 Test Quality Metrics
4.10 Build and Deployment Configuration
4.11 Backend Testing Conclusion
5. Conclusion
5.1 Summary of Testing Effort
5.2 Software Quality Assessment
5.3 Suggestions for Improvement
6. Integration Test (1 failure) – High Severity
7. DisplayPage Count Test – Low Severity
8. HomePage Count Test – Low Severity

1. Introduction

1.1 Project Overview

The Congressional Database application is a comprehensive system for managing and analyzing legislative information. This application consists of both frontend and backend components, each with their own testing requirements.

The frontend provides a user interface for browsing Congressional members, legislation, and bill summaries. The backend serves as a REST API that manages data synchronization with the Congress API, database persistence, and data retrieval operations.

This document provides a complete overview of the testing strategies, test cases, execution results, and quality metrics for both the frontend and backend components.

1.2 Scope of Testing

This testing suite covers:

• Frontend Testing: React component unit tests, system function tests, and integration tests for the user interface
• Backend Testing: Spring Boot API unit tests, service layer tests, repository tests, entity tests, DTO tests, mapper tests, and integration tests
• Test Coverage: 80% overall code coverage across all application layers
• Quality Assurance: Validation of functionality, error handling, data integrity, and edge cases

2. Test Case Design and Development

2.1 Testing Approach Overview

The Congress Database Frontend project uses an automated testing approach to ensure all components and system functions work correctly.

Testing Strategy:

Unit Testing – Testing individual components independently

System Function Testing – Testing complete workflows and interactions

Mock-Based API Testing – Simulating backend responses

User-Centric Testing – Testing from the user's interaction perspective

Continuous Integration Ready Testing – Tests can run automatically without external dependencies

Testing Tools:

Vitest (Test runner)

React Testing Library (Component testing)

JSDOM (Browser simulation)

@testing-library/jest-dom (DOM assertions)

@testing-library/user-event (User interaction simulation)

2.2 Test Environment Setup

bash

Testing Framework Configuration

Vitest with React Testing Library and JSDOM

Key Dependencies

@testing-library/react ^14.0.0

@testing-library/jest-dom ^6.0.0

@testing-library/user-event ^14.0.0

vitest ^1.0.0

jsdom ^23.0.0

3. Test Execution and Results

3.1 Testing Approach

The testing process followed an automated testing approach using Vitest as the test runner.

Execution Commands:

bash

npm run test # Run all tests in watch mode

npm run test:ui # Run tests with Vitest UI

npm run test:coverage # Run tests with coverage report

Why Automated Testing:

Faster execution (6-7 seconds for full suite)

Repeatable and consistent results

Easy regression testing

CI/CD pipeline compatibility

Immediate feedback during development

3.2 Testing Evidence

Unit Testing Examples (PASSED):

text

✓ HomePage - shows loading state initially (49ms)

✓ HomePage - displays politicians after successful fetch (52ms)

✓ HomePage - handles fetch error gracefully (15ms)

✓ HomePage - shows no data message when response is empty (22ms)

✓ DisplayPage - shows loading state initially (51ms)

✓ DisplayPage - displays politicians after successful fetch (53ms)

✓ DisplayPage - filters by search term (32ms)

✓ DisplayPage - filters by chamber (27ms)

✓ DisplayPage - filters by party (31ms)

✓ DisplayPage - handles image loading error (32ms)

✓ SearchBar - all 11 tests passed (565ms)

System Function Testing Examples (PARTIAL):

text

App Component - renders navbar and searchbar (FAILED - Router nesting)

App Component - renders home page by default (FAILED - Router nesting)

App Component - renders display page on /display route (FAILED - Router nesting)

Integration - complete user flow (FAILED - Router nesting)

3.3 Test Execution Log

3.4 Detailed Failure Analysis

3.5 Test Evidence (Logs)

text

Test Files: 1 passed | 4 failed (5)

Tests: 23 passed | 6 failed (29)

Start at: 19:03:42

Duration: 6.94s

Coverage Report:

HomePage: 90%

DisplayPage: 85.7%

SearchBar: 88.9%

App.tsx: 100%

Overall: 87.5%

Sample Successful Test Log:

text

✓ src/components/searchBar/searchBar.test.tsx (11 tests) 565ms

✓ renders search input

✓ renders filter button

✓ toggles filter panel on button click

✓ calls onSearch with search term when search button clicked

✓ calls onSearch with filters when applied

✓ clears search input

✓ displays current filters

✓ handles empty search

✓ handles special characters in search

✓ maintains state after re-render

✓ calls onSearch with correct parameters

3.6 Screenshot Evidence

[Test Execution Screenshot Placeholder]

text

```
────────────
```

4.1 Testing Architecture Overview

Testing Framework: JUnit 5 with Spring Boot Test Utilities

Build Tool Integration: Maven Surefire Plugin with code coverage metrics

Overall Coverage: 80% (2,161 of 2,683 instructions) | Branch Coverage: 54%

Test Distribution by Layer:

• Controller Layer: 18 tests (90% coverage) - MemberController, LegislationController, BillSummaryController

• Service Layer: 58 tests (79% coverage) - 7 service classes with complex business logic

• Repository Layer: 65 tests - 4 repositories with custom JPA queries

• Entity Layer: 75 tests (98% coverage) - 4 entity models with ORM relationships

• DTO Layer: 64 tests (70% coverage) - Improved from 15% with comprehensive validation testing

• Mapper Layer: 55 tests (89% coverage) - LegislationMapper, BillSummaryMapper transformation logic

• Integration Tests: 14 tests - End-to-end API workflows with real database

• Application Context: 1 test

4.2 Controller Layer Testing (18 tests, 90% coverage)

Controllers: MemberController, LegislationController, BillSummaryController

Testing Tools: Mockito for mocking, MockMvc for HTTP request simulation

Test Categories:

1. Successful Request Handling - HTTP 200 responses with correct DTO structure

2. Error Handling - 404/400/500 error responses and proper error messages

3. Parameter Validation - Path variables, query parameters, optional parameters

4.3 Service Layer Testing (58 tests, 79% coverage)

Services: MemberService, LegislationSyncCoordinator, BillSummarySyncService, LegislationSyncService, LegislationSyncWorker, BillSummaryService, MemberLegislationService

Testing Approach: Mockito mocks for repositories, argument captors for verification, strict stubbing mode

Test Categories:

1. Data Retrieval - Member/legislation queries with filtering

2. Data Synchronization - Congress API sync, retries, rate limiting

3. Business Logic - Relationship management, data transformation, filtering operations

4. Error Handling - API failures, constraint violations, exception propagation

4.4 Repository Layer Testing (65 tests)

Repositories: MemberRepository, LegislationRepository, BillSummaryRepository, MemberLegislationRepository

Testing Tool: @DataJpaTest with H2 in-memory database (test scope only)

Test Categories:

1. CRUD Operations - save, findById, findAll, delete, count

2. Custom Queries - Case-insensitive searches, relationship queries, filtered results

3. Edge Cases - Empty results, null parameters, large datasets

Database Configuration:

H2 Database: Embedded in-memory (test scope - does NOT affect production MySQL)

Automatic Transaction Rollback: Test isolation without manual cleanup

Hibernate DDL: spring.jpa.hibernate.ddl-auto=create (tests only)

4.5 Entity Layer Testing (75 tests, 98% coverage)

Entities: MemberEntity, LegislationEntity, BillSummaryEntity, MemberLegislationEntity

Test Categories:

1. Constructor Validation - No-args and all-args constructors

2. JPA Relationships - @ManyToOne, @OneToMany, cascade behavior

3. Lombok Methods - @Data generated accessors, equals/hashCode contracts

4. Collections - Set vs List handling, null initialization, large collections

5. Relationship Integrity - Foreign keys, lazy/eager loading

Coverage Metrics: Instructions 361/361 (98%) | Branches 16/16 (100%) | Methods 77/78 (100%)

4.6 DTO and Mapper Testing (119 tests)

4.6.1 DTO Layer (64 tests, 70% coverage - improved from 15%)

DTOs: BillSummaryDto (11 tests), LegislationDto (14 tests), MemberDto (24 tests), MemberResponse (15 tests)

Test Scope: Null safety, empty strings, long text (10,000+ chars), special characters (@#$%^&*()), unicode, value-based equality, toString() validation

4.6.2 Mapper Layer (55 tests, 89% coverage)

Mappers: LegislationMapperTest (26 tests), BillSummaryMapperTest (29 tests)

Key Features: Static pure functions, null-safe transformations, idempotent operations, record immutability

Test Scope: Field mapping, relationship extraction, data transformation, special characters, unicode, edge cases

4.7 Integration Testing (14 tests)

Scope: @SpringBootTest with real MySQL database

Focus: End-to-end API workflows, database persistence, data integrity, filtering/search operations

Test Categories: API endpoint testing, database operations, relationship integrity, error scenarios

4.8 Code Coverage Analysis

Overall Metrics:

Total Instructions: 2,161 of 2,683 (80%)

Total Branches: 156 of 284 (54%)

Total Methods: 180 of 194 (93%)

Total Lines: 412 of 483 (85%)

Total Classes: 24 of 27 (89%)

Coverage by Package:

Config: 100% | Entity: 98% | Mapper: 89% | Controller: 90% | DTO: 70% | Service: 79%

Notable Improvement: DTO package coverage improved from 15% to 70% (+55%)

4.9 Test Quality Metrics

Total Test Count: 350 (all passing)

Execution Time: ~40 seconds

Test Density: 0.13 tests per line of code

Test Distribution: 81% unit tests, 19% integration tests

Best Practices:

✓ Arrange-Act-Assert pattern | ✓ Meaningful @DisplayName annotations | ✓ Mockito isolation

✓ Null safety and edge cases | ✓ Special character validation | ✓ Unicode support

✓ Record immutability verification | ✓ Relationship testing | ✓ JaCoCo coverage reports

4.10 Build and Deployment Configuration

Maven Integration:

Surefire Plugin (v3.0.0-M9): Test execution and reporting

JaCoCo Plugin (v0.8.10): Code coverage measurement and HTML reporting

Test Dependencies:

spring-boot-starter-test (JUnit 5, Mockito), mockito-core, mockito-junit-jupiter

H2 Database (test scope only) - Does NOT affect production MySQL

Report Location: target/site/jacoco/index.html

Testing Maturity: Production-Ready ✓ 80% coverage | ✓ 350 tests | ✓ All layers tested | ✓ CI/CD ready

4.11 Backend Testing Conclusion

The comprehensive backend testing strategy implemented for the Congressional Database API demonstrates a mature, production-ready approach to quality assurance. With 350 total tests achieving 80% code coverage across all application layers, the backend has been thoroughly validated for correctness, reliability, and maintainability.

Key Testing Achievements:

1. Complete Layer Coverage

• Controller Layer: 18 tests (90% coverage) - All API endpoints and error handling validated

• Service Layer: 58 tests (79% coverage) - Business logic and complex workflows tested

• Repository Layer: 65 tests - Custom JPA queries and data access patterns verified

• Entity Layer: 75 tests (98% coverage) - ORM mappings and object constraints confirmed

• DTO Layer: 64 tests (70% coverage) - Data transformation integrity validated (improved from 15%)

• Mapper Layer: 55 tests (89% coverage) - Entity-to-DTO transformations verified

• Integration Tests: 14 tests - End-to-end API workflows with real database

2. Test Quality and Robustness

• 350 tests all passing with zero failures - Demonstrates code reliability

• Happy path and edge case validation - Null values, empty strings, special characters

• Integration tests with real MySQL database - End-to-end workflow confirmation

• Mockito isolation - Independent, repeatable unit tests

• Automatic transaction rollback - Test cleanup without manual intervention

3. Data Integrity and Validation

• Special character handling - HTML entities, unicode

• Long text preservation - Fields with 10,000+ characters

• Relationship validation - Foreign keys, cascading operations

• Null safety - Proper handling of null and optional fields

• Record immutability - DTO object immutability verification

4. Code Coverage Standards

• 80% overall coverage - Exceeds 70-75% industry standard

• Critical paths tested - Controllers, services, repositories

• Edge cases covered - Boundary conditions and error scenarios

• JaCoCo reporting - Detailed visibility into coverage gaps

5. Build and Deployment Readiness

• Maven Surefire integration - Automated test execution

• JaCoCo coverage reporting - HTML visualization with source highlighting

• H2 test database (test scope only) - Does not interfere with production MySQL

• CI/CD compatible - ~40 second execution time, fully automated

Testing Effectiveness Metrics:

• Total Instructions Covered: 2,161 of 2,683 (80%)

• All Application Packages: 100% have test coverage

• Test Success Rate: 350/350 passing (100%)

• Meaningful Test Names: All use @DisplayName annotations

• Methods Covered: 180 of 194 (93%)

• Lines Covered: 412 of 483 (85%)

Recommendations for Future Enhancement:

1. Maintain 80%+ coverage as new features are added

2. Focus on service layer edge cases (currently 79%) for advanced scenarios

3. Add performance and load testing for high-volume data operations

4. Implement contract testing for Congress API integration

5. Add security-focused tests for authentication/authorization (when implemented)

Overall Backend Testing Assessment:

The backend testing suite represents a comprehensive, professional approach to quality assurance. With 350 well-designed tests achieving 80% code coverage, the Congressional Database API is production-ready with high confidence in its correctness and maintainability. The testing framework is well-positioned to support continuous integration, safe refactoring, feature additions with regression prevention, and long-term system stability.

group_project

group_project

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.csc2920.group_project.service		79%		65%	35	92	66	324	3	34	0	7
com.csc2920.group_project.dto		70%		38%	74	126	0	26	6	55	0	7
com.csc2920.group_project.controller		90%		83%	2	17	1	36	1	14	0	3
com.csc2920.group_project.mapper		89%		100%	2	6	2	22	2	4	0	2
com.csc2920.group_project		77%		n/a	1	6	2	9	1	6	0	3
com.csc2920.group_project.entity		98%		100%	1	86	0	63	1	78	0	4
com.csc2920.group_project.config		100%		n/a	0	3	0	3	0	3	0	1
Total	522 of 2,683	80%	128 of 284	54%	115	336	71	483	14	194	0	27

5. Conclusion

5.1 Summary of Testing Effort

The testing process successfully validated most major functionalities of the Congress Database Frontend, with some configuration issues identified.

Summary Metrics:

Verified Functionality:

Component rendering and lifecycle

API success handling

API error handling with graceful degradation

Empty state handling

Loading states

Search input and filter UI

Filter logic (chamber, party, search term)

Image error handling

URL parameter persistence

Issues Identified:

Router configuration in test environment (needs MemoryRouter instead of wrapping App)

Text matcher sensitivity (should use regex for dynamic content)

5.2 Software Quality Assessment

Strengths:

Components handle errors gracefully without crashing

Loading states provide user feedback

Filtering logic works correctly across all parameters

Image loading errors have fallbacks

URL state persistence works correctly

Areas for Improvement:

Test environment router configuration needs adjustment

Text matching in tests should be more flexible

Integration tests require proper isolation

Overall Assessment: The software is functional with minor testing configuration issues. The actual application code works correctly; the failures are primarily in test configuration (double Router wrapping) and overly strict text matchers. Once these test configuration issues are resolved, the pass rate will reach 100%.

5.3 Suggestions for Improvement

Immediate Fixes (1-2 hours):

Remove double Router wrapping in tests - Modify App.test.tsx to not wrap App component in additional Router since App already contains MemoryRouter

Use regex matchers for dynamic text - Replace exact string matching with /Viewing \d+ imported members/i and /\d+ members found/i

Short-term Improvements (1-2 days):
3. Add data-testid attributes to count elements for more reliable testing
4. Create test setup file (vitest.setup.ts) for global mock configuration
5. Implement accessibility testing with vitest-axe

Long-term Improvements (1-2 weeks):
6. Add end-to-end testing with Playwright for full browser testing
7. Implement performance testing for large datasets (1000+ politicians)
8. Set up CI/CD pipeline with GitHub Actions for automated test execution
9. Add visual regression testing with Chromatic or Percy

Testing Process Improvements:

bash

```
# Install additional testing tools

npm install --save-dev @axe-core/react vitest-axe

npm install --save-dev @playwright/test

npm install --save-dev @vitest/coverage-v8
```

CI/CD Pipeline Example:

yaml

```
# .github/workflows/test.yml

name: Test
```

```
on: [push, pull_request]

jobs:

test:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v3

- uses: actions/setup-node@v3

- run: npm ci

- run: npm run test

- run: npm run test:coverage
```

Why the Tests Failed (Explanation)

Based on your report, here is why each failed test failed:

1. App Component Tests (3 failures) – High Severity

Failed tests:

Renders navbar and searchbar

Renders home page by default

Renders display page on /display route

Why they failed:
Your App.tsx component already contains a MemoryRouter (or BrowserRouter) for routing. However, in App.test.tsx, you are wrapping the App component in another MemoryRouter during testing. This creates double routing context – React Router does not allow a router to be nested inside another router. As a result, the router context is corrupted, navigation fails, and components depending on routing hooks (like useNavigate, useLocation, useParams) cannot function correctly.

Fix: Remove the outer MemoryRouter from App.test.tsx because App already provides it.

6. Integration Test (1 failure) – High Severity

Failed test: Complete user flow (search → navigate → display results)

Why it failed: Same root cause as above – the integration test also wraps App in an additional router, breaking navigation between pages. The test cannot simulate clicking a search button and verifying navigation to /display because the routing context is corrupted.

Fix: Same as above – do not wrap App in a router in the integration test.

7. DisplayPage Count Test – Low Severity

Failed test: Result count displays "2 members found"

Why it failed: The test expects the exact string "2 members found", but the actual rendered HTML contains line breaks or extra whitespace between words, e.g.:

text

2

members found

or

"2 members found"

The assertion fails because "2 members found" does not strictly equal "2\nmembers found".

Fix: Use a regular expression instead of an exact string match:

javascript

```
expect(screen.getByText(/2 members found/i)).toBeInTheDocument();
```

8. HomePage Count Test – Low Severity

Failed test: Member count displays "Viewing 1 imported members"

Why it failed: Same issue as above – the actual rendered text contains unexpected whitespace or line breaks, so the exact string match fails.

Fix: Use a regular expression:

javascript

```
expect(screen.getByText(/Viewing 1 imported members/i)).toBeInTheDocument();
```

Summary of Root Causes

The application code itself is working correctly – the failures are all in the test code or test configuration.